# Combining Small Language Models with LLM-as-a-Judge for Code Smell Detection

Saymon Souza

PhD. Student

UFMG
UNIVERSIDADE FEDERAL
DE MINAS GERAIS

lab-soft
software engineering laboratory

# Content

- EMSE Paper Recap
- FORGE Paper
  - Motivation
  - Methodology
  - Research Questions
  - Results Discussion
- Next steps

# What We Learned So Far

# EMSE Paper Recap

- We learned that LLMs **can be valuable contributors** as code smell detection tools.
- We also learned that they are best suited to **detect simpler smells** (i.e., Long Method, Large Class, Data Class)
- Finally, we learned that at least for now **they are not viable replacements** for static analysis tools.

# Good, but is it viable?

- Even if LLMs performed better than SATs at detecting code smells in all scenarios, one could ask:
  - Would their use be even viable in a real world scenario?



# LLMs are expensive

- Gemma 3, 4B-parameter instruction-tuned model, can be run on a consumer grade Nvidia RTX 3060 GPU with 12GB of memory (~$300) [1].

- For even larger models, such as Llama-3.3-70B-Instruct, inference requires three A100 80GB GPUs, totaling about $50.000 [1].



[1] Crupi, Giuseppe, Tufano, Rosalia and Bavota, Gabriel . "***Improving code generation via small language model-as-a-judge.***" International Conference on Software Engineering (ICSE), 2026.

# Where to go next?

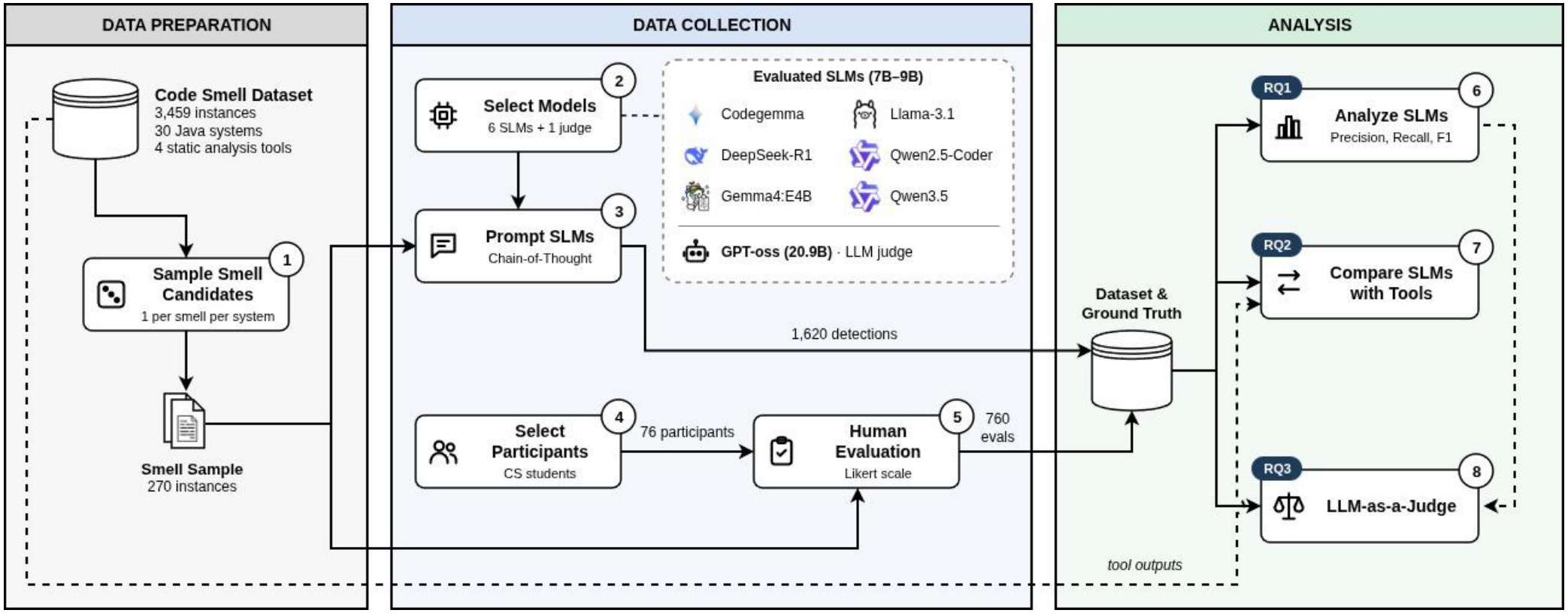
# Research Goal

*Analyze a multi-model framework that combines SLMs with LLM arbitration (LLM-as-a-Judge), for the purpose of evaluating its code smell detection performance, with respect to effectiveness and conflict resolution, from the point of view of software engineering researchers and developers, in the context of automated code smell detection in Java source code.*

# Research Questions

- RQ1: How effective are SLMs in detecting code smells?
- RQ2: How do SLMs compare with static analysis tools in code smell detection?
- RQ3: How effective is the LLM-as-a-Judge in resolving conflicts between SLMs?

# Methodology



# Studied Smells

- We analyzed nine code smells:
  - Class-level: Large Class, Data Class and Refused Bequest;
  - Method-level: Feature Envy, Intensive Coupling, Dispersed Coupling, Long Parameter List, Shotgun Surgery and Long Method.

# Chosen SLMs and LLM

- We chose 6 SLMs and 1 LLM for our experiment:
  - Qwen3.5 with 9B parameters
  - Deepseek-R1 with 8B parameters
  - Llama 3.1 with 8B parameters
  - Gemma4:E4B with 8B parameters
  - Qwen2.5-Coder with with 7B parameters
  - Google's Codegemma with 7B parameters
  - GPT-oss with 20B parameters for the **LLM-as-a-judge**

lab-soft
software engineering laboratory

# Experiment Setup

- All models ran with temperature 0.0 and *top_p* 0.95.
- All models used the same prompt for each smell.
- We used Ollama (version 0.32.13) as interface between the prompt and the models.

# PC Specs for the Experiment

- We ran our experiment on a PC with these configurations:
  - OS: Ubuntu 26.04 LTS
  - Processor: AMD Ryzen 5 7600
  - Memory: 32 GB DDR5 / 5600 MT/s
  - GPU: Nvidia RTX 4070 12 GB
  - SSD: 2TB

# References for the prompt

- We used the detections strategies present in Lanza and Marinescu's book for 8 of the 9 smells in the study:
  - Lanza. Michele. and Radu Marinescu. "Object-oriented metrics in practice: using software metrics to characterize. evaluate and improve the design of object-oriented systems." 2006.
- For LPL we used the following paper that brings some detection tips:
  - Bhave. Anushka. and Roopak Sinha. *"Deep multimodal architecture for detection of long parameter list and switch statements using distilbert."* 2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM). 2022.

# Detection strategies

- Each smell in Lanza and Marinescu's book has a dedicated chapter with a "Detection" section that explains how to detect the smell.

- We used the paragraphs of this section to create the "step-by-step" guide that the models should follow.

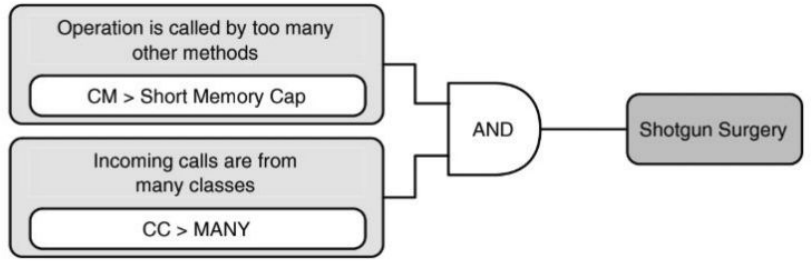



**Fig. 6.14.** Shotgun Surgery *detection strategy*

Based on all the considerations above, the detection technique is now easy to describe (see Fig. 6.14). First, we pick up those functions that have a *strong* change impact, and from these we keep only those that also have a high *dispersion* of changes. The *detection strategy* in detail is:

1. **Operation is called by too many other operations.** When a change in the measured operation occurs we must fix all the other operations that depend on it. If this exceeds our short-term memory capacity the risk of missing a dependency increases This justifies both the selection of the metric and of the threshold.
   An alternative way to quantify the strength of incoming dependencies is to count the *number of calls* instead of *the number of callers* (like CM (Changing Methods) does). The metric called *Weighted Changing Method* (WCM) defined it [Mar02a] does just that.
2. **Incoming calls are from many classes.** Using this metric and

# Prompt

**PROMPT TEMPLATE** · Large Class

**ROLE**

You are a software expert analyzing one Java file for symptoms that may indicate the Large Class code smell.

**DEFINITION**

A Large Class is a class that is excessively large and complex, has low cohesion, and heavily accesses data from other classes, often centralizing too much functionality and responsibility.

**QUESTIONS**

Please answer the following questions step by step before reaching a conclusion about the code smell:

1. Does the class appear to centralize too much functionality or responsibility, acting as a controller or "brain" for a large part of the system?
2. Does the class have low cohesion (i.e., its methods do not work together, or there are many unrelated responsibilities)?
3. Does the class heavily access data from other classes (i.e., frequently accesses fields or methods of other classes, either directly or via accessors)?
4. Is the class large and complex (i.e., has many methods, many fields, or high overall complexity)?

**OUTPUT FORMAT**

Respond only with the JSON below, without markdown or extra text. The justification must have at most 3 sentences citing concrete evidence from the file (method/field names).

```
{"criterion": "large_class",
 "guidelines": [{
   "id": "main_class",
   "overall_assessment": "YES | NO",
   "confidence": "1-5",
   "justification": "<evidence-based explanation>"}]}
```

*confidence scale*

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| definitely no smell | probably no smell | uncertain | probably smell | definitely smell |

**INPUT**

```
[Java source code with the smell]
```

**Literature basis (not part of the prompt)**
Lanza and Marinescu, Section 5.3

- *"God Class refers to classes that tend to centralize the intelligence of the system."*
- *"They have a lot of non-communicative behavior i.e., there is a low cohesion between the methods belonging to that class."*
- *"They heavily access data of other simpler classes, either directly or using accessor methods."*
- *"A God Class performs too much work on its own, delegating only minor details to a set of trivial classes and using the data from other classes."*

# Results - RQ1

# Data Class

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Qwen3.5 | 9B | 0.88 | 0.82 | 0.85 |
| Gemma4:E4B | 8B | 0.79 | 0.88 | 0.83 |
| **Deepseek-R1** | **8B** | **0.93** | **0.82** | **0.88** |
| Llama3.1 | 8B | 0.78 | 0.82 | 0.80 |
| Codegemma | 7B | 0.57 | 1.00 | 0.72 |
| Qwen2.5-Coder | 7B | 0.59 | 1.00 | 0.74 |

# Long Method

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Qwen3.5 | 9B | 1.00 | 0.48 | 0.65 |
| Gemma4:E4B | 8B | 0.95 | 0.76 | 0.84 |
| Deepseek-R1 | 8B | 0.94 | 0.64 | 0.76 |
| **Llama3.1** | **8B** | **0.86** | **0.96** | **0.91** |
| Codegemma | 7B | 0.83 | 1.00 | 0.91 |
| Qwen2.5-Coder | 7B | 0.81 | 0.88 | 0.85 |

# Shotgun Surgery

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Qwen3.5 | 9B | 1.00 | 0.19 | 0.32 |
| Gemma4:E4B | 8B | 0.50 | 0.44 | 0.47 |
| Deepseek-R1 | 8B | 0.75 | 0.56 | 0.64 |
| Llama3.1 | 8B | 0.52 | 0.88 | 0.65 |
| **Codegemma** | **7B** | **0.54** | **0.94** | **0.68** |
| Qwen2.5-Coder | 7B | 0.56 | 0.31 | 0.40 |

# Refused Bequest

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Qwen3.5 | 9B | 0.00 | 0.00 | 0.00 |
| Gemma4:E4B | 8B | 0.29 | 0.50 | 0.37 |
| Deepseek-R1 | 8B | 0.57 | 0.40 | 0.47 |
| **Llama3.1** | **8B** | **0.38** | **0.80** | **0.52** |
| Codegemma | 7B | 0.33 | 1.00 | 0.50 |
| Qwen2.5-Coder | 7B | 0.50 | 0.30 | 0.38 |

# How effective are SLMs in detecting smells?

- SLMs can detect several code smells effectively, but their performance is highly smell-dependent.
- SLMs tend to prioritize recall.
- No single SLM dominates across all code smells.

# Results - RQ2

# Data Class

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Organic | - | 0.67 | 0.71 | 0.69 |
| PMD | - | 0.70 | 0.82 | 0.76 |
| Qwen3.5 | 9B | 1.00 | 0.48 | 0.65 |
| Gemma4:E4B | 8B | 0.95 | 0.76 | 0.84 |
| Deepseek-R1 | 8B | 0.94 | 0.64 | 0.76 |
| **Llama3.1** | **8B** | **0.86** | **0.96** | **0.91** |
| Codegemma | 7B | 0.83 | 1.00 | 0.91 |
| Qwen2.5-Coder | 7B | 0.81 | 0.88 | 0.85 |

# Long Method

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| JDeodorant | - | 0.83 | 0.76 | 0.79 |
| Organic | - | 1.00 | 0.72 | 0.84 |
| Qwen3.5 | 9B | 1.00 | 0.48 | 0.65 |
| Gemma4:E4B | 8B | 0.95 | 0.76 | 0.84 |
| Deepseek-R1 | 8B | 0.94 | 0.64 | 0.76 |
| **Llama3.1** | **8B** | **0.86** | **0.96** | **0.91** |
| Codegemma | 7B | 0.83 | 1.00 | 0.91 |
| Qwen2.5-Coder | 7B | 0.81 | 0.88 | 0.85 |

# Long Parameter List

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| **Organic** | **-** | **0.57** | **1.00** | **0.73** |
| PMD | - | 0.67 | 0.25 | 0.36 |
| Qwen3.5 | 9B | 0.70 | 0.44 | 0.54 |
| Gemma4:E4B | 8B | 0.61 | 0.69 | 0.65 |
| Deepseek-R1 | 8B | 0.52 | 0.88 | 0.65 |
| **Llama3.1** | **8B** | **0.55** | **1.00** | **0.71** |
| Codegemma | 7B | 0.56 | 0.94 | 0.70 |
| Qwen2.5-Coder | 7B | 0.60 | 0.38 | 0.46 |

# Shotgun Surgery

| Evaluator | Parameters | Precision | Recall | F1-Score |
|---|---|---|---|---|
| **Jspirit** | **-** | **0.63** | **0.75** | **0.69** |
| Organic | - | 0.50 | 0.44 | 0.47 |
| Qwen3.5 | 9B | 1.00 | 0.19 | 0.32 |
| Gemma4:E4B | 8B | 0.50 | 0.44 | 0.47 |
| Deepseek-R1 | 8B | 0.75 | 0.56 | 0.64 |
| Llama3.1 | 8B | 0.52 | 0.88 | 0.65 |
| **Codegemma** | **7B** | **0.54** | **0.94** | **0.68** |
| Qwen2.5-Coder | 7B | 0.56 | 0.31 | 0.40 |

# SLMs generally outperform the SATs

| Code Smell | Best SAT | Best F1 between SATs | Best SLM | Best F1 between SLMs | Difference |
|---|---|---|---|---|---|
| Data Class | PMD | 0.76 | Gemma4:E4B (8B) | 0.88 | +0.12 |
| Dispersed Coupling | Jspirit | 0.73 | Llama3.1 (8B) | 0.77 | +0.04 |
| Feature Envy | Organic | 0.62 | Codegemma (7B) | 0.74 | +0.12 |
| Intensive Coupling | Jspirit | 0.67 | Gemma4:E4B (8B) | 0.78 | +0.11 |
| Large Class | Jspirit | 0.73 | Deepseek-R1 (8B) | 0.85 | +0.12 |
| Long Method | Organic | 0.84 | Llama3.1 (8B) | 0.91 | +0.07 |
| Long Parameter List | Organic | 0.73 | Deepseek-R1 (8B) | 0.71 | -0.02 |
| Refused Bequest | Jspirit | 0.45 | Codegemma (7B) | 0.52 | +0.07 |
| Shotgun Surgery | Jspirit | 0.69 | Llama3.1 (8B) | 0.68 | -0.01 |

# How do SLMs compare with SATs?

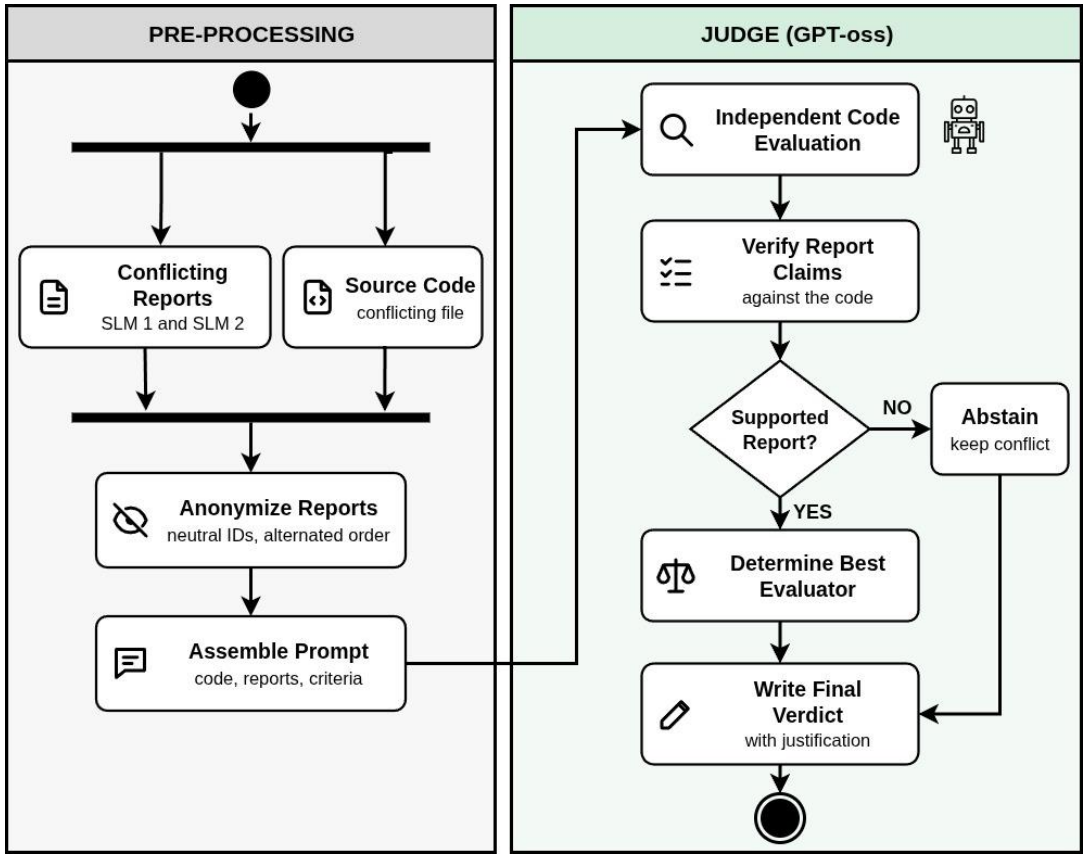
- SLMs are generally better, but not always substantially better.
  - Especially when a precise structural rule is already effective (i.e. Long Param. List).

- SLMs and static tools exhibit different error profiles.

- Difficult smells remain difficult for both approaches.

# Results - RQ3

# How the judge works?



# Conflicts Resolved by the Judge (1st vs 2nd/3rd)

| Smell | Pair | Conflicts | Hits |
|---|---|---|---|
| Data Class | *(1st)* DeepSeek vs Qwen *(2nd)*<br>*(1st)* DeepSeek vs Qwen *(3rd)* | 7<br>4 | 6 (86%)<br>3 (75%) |
| Dispersed Coupling | *(1st)* CodeGemma vs Llama *(2nd)*<br>*(1st)* CodeGemma vs Qwen-Coder *(3rd)* | 2<br>7 | 1 (50%)<br>4 (71%) |
| Feature Envy | *(1st)* CodeGemma vs DeepSeek *(2nd)*<br>*(1st)* CodeGemma vs Llama *(3rd)* | 9 *(1 abstention)*<br>8 *(1 abstention)* | 7 (87%)<br>4 (57%) |
| Intensive Coupling | *(1st)* CodeGemma vs Llama *(2nd)*<br>*(1st)* CodeGemma vs Gemma *(3rd)* | 2<br>11 | 1 (50%)<br>5 (45%) |
| Large Class | *(1st)* Qwen-Coder vs Llama *(2nd)*<br>*(1st)* Qwen-Coder vs Gemma *(3rd)* | 3<br>5 | 2 (67%)<br>3 (60%) |
| Long Method | *(1st)* Llama vs CodeGemma *(2nd)* *(1st)*<br>Llama vs Qwen-Coder *(3rd)* | 2<br>3 | 1 (50%)<br>2 (67%) |
| Long Parameter List | *(1st)* Llama vs CodeGemma *(2nd)*<br>*(1st)* Llama vs Gemma *(3rd)* | 4<br>11 | 3 (75%)<br>7 (64%) |
| Refused Bequest | *(1st)* Llama vs CodeGemma *(2nd)*<br>*(1st)* Llama vs DeepSeek *(3rd)* | 9<br>18 | 6 (67%)<br>12 (67%) |
| Shotgun Surgery | *(1st)* CodeGemma vs Llama *(2nd)*<br>*(1st)* CodeGemma vs DeepSeek *(3rd)* | 5<br>18 | 4 (80%)<br>9 (50%) |
| | | **128 (126 veredicts)** | **80 (63%)** |

lab-soft
software engineering laboratory

# Conflicts Resolved by the Judge (1st vs 2nd/6th)

| Smell | Pair | Conflicts | Hits |
|---|---|---|---|
| Data Class | *(1st)* DeepSeek vs Qwen *(2nd)*<br>*(1st)* DeepSeek vs Qwen *(6th)* | 7<br>14 | 6 (86%)<br>12 (85%) |
| Dispersed Coupling | *(1st)* CodeGemma vs Llama *(2nd)*<br>*(1st)* CodeGemma vs Gemma *(6th)* | 2<br>28 *(3 abstentions)* | 1 (50%)<br>14 (50%) |
| Feature Envy | *(1st)* CodeGemma vs DeepSeek *(2nd)*<br>*(1st)* CodeGemma vs Qwen *(6th)* | 9<br>22 *(4 abstentions)* | 6 (67%)<br>7 (38%) |
| Intensive Coupling | *(1st)* CodeGemma vs Llama *(2nd)*<br>*(1st)* CodeGemma vs Gemma *(6th)* | 2<br>26 *(3 abstentions)* | 1 (50%)<br>11 (47%) |
| Large Class | *(1st)* Qwen-Coder vs Llama *(2nd)*<br>*(1st)* Qwen-Coder vs Gemma *(6th)* | 3<br>16 | 2 (67%)<br>10 (62%) |
| Long Method | *(1st)* Llama vs CodeGemma *(2nd)*<br>*(1st)* Llama vs Qwen *(6th)* | 2<br>16 | 1 (50%)<br>7 (43%) |
| Long Parameter List | *(1st)* Llama vs CodeGemma *(2nd)*<br>*(1st)* Llama vs Qwen-Coder *(6th)* | 4<br>19 | 3 (75%)<br>9 (47%) |
| Refused Bequest | *(1st)* Llama vs CodeGemma *(2nd)*<br>*(1st)* Llama vs Qwen *(6th)* | 9<br>21 | 6 (67%)<br>13 (62%) |
| Shotgun Surgery | *(1st)* CodeGemma vs Llama *(2nd)*<br>*(1st)* CodeGemma vs Qwen *(6th)* | 5<br>25 | 4 (80%)<br>14 (56%) |
| | | **230 (220 veredicts)** | **127 (57%)** |

lab-soft
software engineering laboratory

# Best SLM vs Best Pair of The LLM-as-a-Judge

| | Best SLM | | | Best LLM-as-a-Judge | | |
|---|---|---|---|---|---|---|
| **Smell** | **Precision** | **Recall** | **F1-Score** | **Precision** | **Recall** | **F1-Score** |
| Data Class | 0.93 | 0.82 | 0.88 | **0.94** | **0.94** | **0.94** |
| Dispersed Coupling | 0.62 | **1.00** | 0.77 | **0.68** | 0.94 | **0.79** |
| Feature Envy | 0.64 | 0.88 | 0.74 | **0.68** | **0.94** | **0.79** |
| Intensive Coupling | 0.63 | 1.00 | 0.78 | 0.63 | 1.00 | 0.78 |
| Large Class | 0.74 | **1.00** | **0.85** | **0.76** | 0.95 | 0.84 |
| Long Method | 0.86 | 0.96 | 0.91 | 0.86 | 0.96 | 0.91 |
| Long Parameter List | 0.55 | **1.00** | 0.71 | **0.58** | 0.94 | 0.71 |
| Refused Bequest | 0.38 | 0.80 | 0.52 | 0.38 | **0.90** | **0.53** |
| Shotgun Surgery | 0.54 | 0.94 | 0.68 | **0.70** | 0.94 | **0.70** |

# How effective is the LLM-as-a-Judge?

- The LLM-as-a-Judge resolves 62.5% of the conflicts correctly, a modest but statistically significant advantage over random choice.
- The LLM-as-a-Judge achieves the best F1 for 5 out of the 9 smells compared with the best SLM.
- The judge mostly acts as a filter for false positives, often trading Recall for Precision.

# Next Steps

- Submit the paper to FORGE
- Work on ICPC - RENE paper
  - Differentiated replication from a CAIN'25 paper
- Work on the evolution of the FORGE paper
  - Improve our ground truth by experienced professionals evaluate the reports produced by the SLMs and LLM-as-a-judge

# Thank you!

*Saymon Souza | silvasouzasaymon@gmail.com*